

NodeJS – Best Practices

v1.2.0

26-October-2026

CO-CREATE TO OUTPERFORM | WNS

Prepared By:	Adarsh Naik
Reviewed By:	Nikitha Amaresh
Approved By:	Vivek Kumar

Version	Date	Change Mode (C/A/M/D)	Description of change	Prepared/ Revised by	Reviewed By	Approved By
1.0.0	06/02/2024	C	First version	Adarsh Naik	Nikitha Amaresh	Vivek Kumar
1.1.0	28/06/2024	A	2.2.12 Use of try-catch in sync code only	Adarsh Naik	Nikitha Amaresh	Vivek Kumar
1.2.0	26/10/2026	A	Added 8. HTTP Timeouts and Resource Limits, 9. Graceful Shutdown, 10. Distributed Tracing and Standardized Metrics	Pending	Pending	Pending

C – Created, A – Added, M – Modified, D – Deleted

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

Contents

1. Introduction.....1

2. Coding Guidelines……...3

3. Application Prerequisite.....29

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

1. Introduction

Node.js is an open-source backend JavaScript runtime environment. It is used as backend service where JavaScript works on the server-side of the application. This way JavaScript is used on both frontend and backend. Node.js runs on chrome v8 engine which converts JavaScript code into machine code, it is highly scalable, lightweight, fast, and data intensive.

Working of Node.js: Node.js accepts the request from the clients and sends the response, while working with the request node.js handles them with a single thread. To operate I/O operations or requests node.js use the concept of threads. Thread is a sequence of instructions that the server needs to perform. It runs parallel on the server to provide the information to multiple clients. Node.js is an event loop single-threaded language. It can handle concurrent requests with a single thread without blocking it for one request.

Node.js basically works on two concepts.

- Asynchronous
- Non-blocking I/O

Non-blocking I/o: Non-blocking I/O means working with multiple requests without blocking the thread for a single request. I/O basically interacts with external systems such as files, databases. Node.js is not used for CPU-intensive work means for calculations, video processing because a single thread cannot handle the CPU works.

Asynchronous: Asynchronous is executing a call-back function. The moment we get the response from the other server or database it will execute a call back function. Call-back functions are called as soon as some work is finished, and this is because the node.js uses an event-driven architecture. The single thread doesn't work with the request instead it sends the request to another system which resolves the request, and it is accessible for another request.

To implement the concept of the system to handle the request node.js uses the concept of Libuv.

Libuv is an open-source library built-in C. It has a strong focus on asynchronous and I/O, this gives node access to the underlying computer operating system, file system, and networking.

Libuv implements two extremely important features of node.js.

- Event loop
- Thread pool

Event loop: The event loop contains a single thread and is responsible for handling easy tasks like executing call-backs and network I/O. When the program is to initialize all the top-level code is executed, the code is not in the call-back function. All the applications code that is inside call-back functions will run in the event loop. Event Loop is the heart of node.js. When we start our node application the event loop starts running right away. Most of the work is done in the event loop.

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

Nodejs use event-driven-architecture.

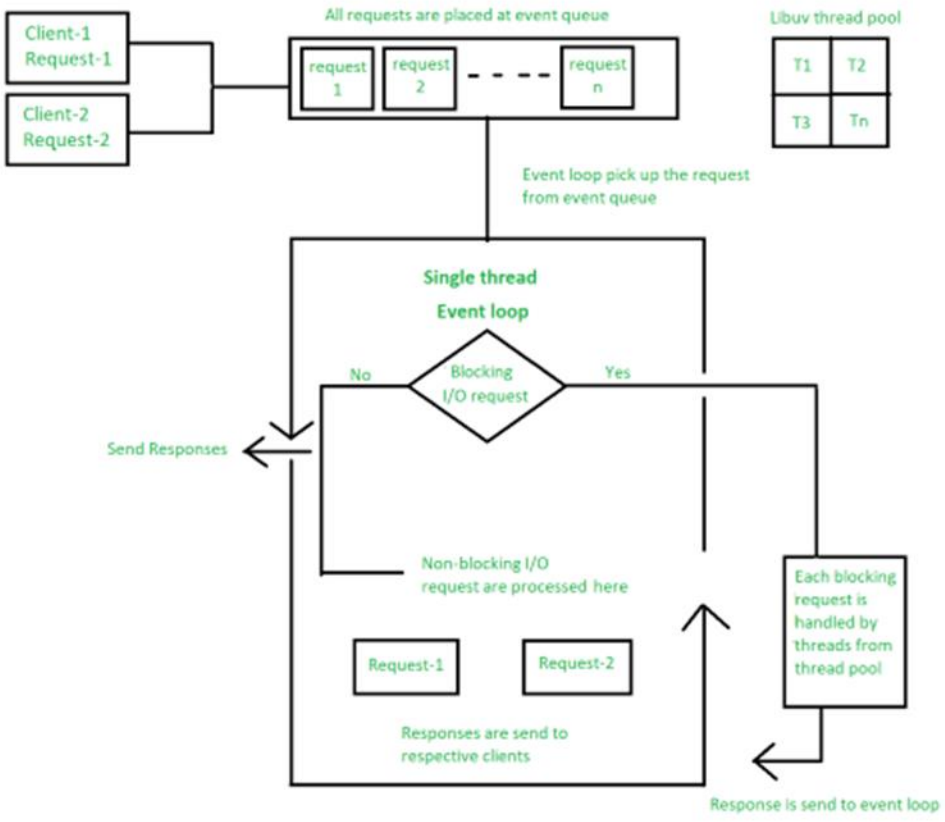
- Events are emitted.
- Event loop picks them up.
- Call-backs are called.

Event queue: As soon as the request is sent the thread places the request into a queue. It is known as an event queue. The process like app receiving HTTP request or server or a timer will emit event as soon as they are done with the work and event loop will pick up these events and call the call-back functions that are associated with each event and response is sent to the client.

The event loop is an indefinite loop that continuously receives the request and processes them. It checks the queue and waits for the incoming request indefinitely.

Thread pool: Though node.js is single threaded it internally maintains a thread pool. When non-blocking requests are accepted there are processed in an event loop, but while accepting blocking requests it checks for available threads in a thread pool, assigns a thread to the client's request which is then processed and send back to the event loop, and response is sent to the respective client.

The thread pool size can be change:



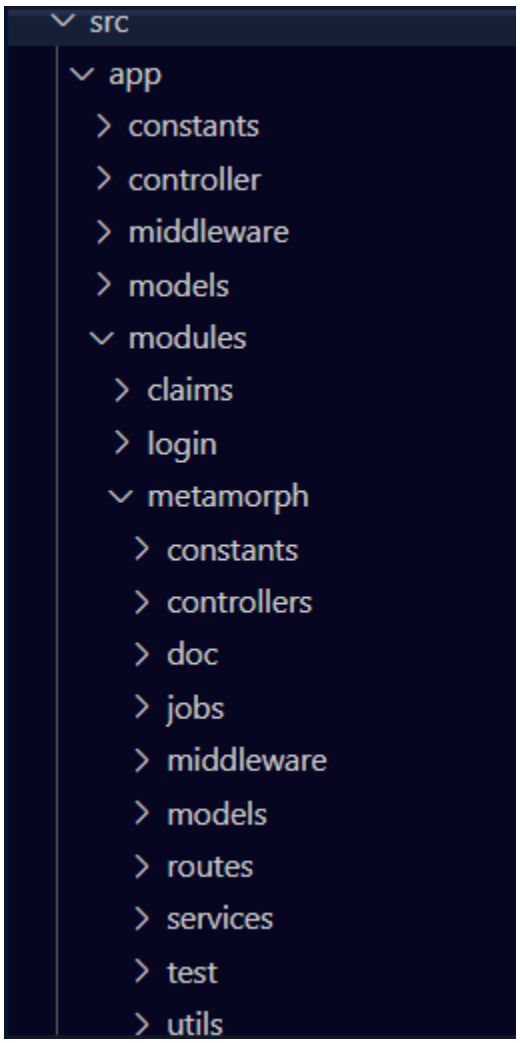
© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

2. Coding Guidelines

Following best practices in NodeJS would enable us to deliver a performant web application.

1. Project Architecture Practices

Having a well-defined folder structure (modular approach) reduces maintenance bottlenecks by reducing complexity.



Creating a new project

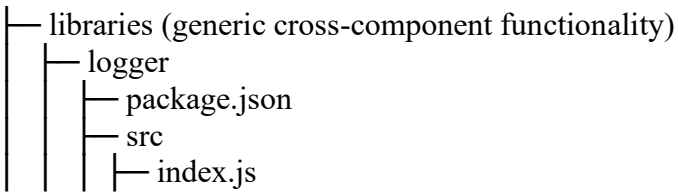
Use npm init to create a new project. It will automatically generate a package.json

Separation of concerns is important. It's better to have API calls in a separate file, to keep it clean and avoid confusion.

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

1.1 Wrap common utilities as packages, consider publishing

Place all reusable modules in a dedicated folder, e.g., "libraries", and underneath each module in its own folder, e.g., "/libraries/logger". Make the module an independent package with its own package.json file to increase the module encapsulation and allows future publishing to a repository. In a Monorepo setup, modules can be consumed by 'npm linking' to their physical paths, using ts-paths or by publishing and installing from a package manager repository like the npm registry. Otherwise, Clients of a module might import and get coupled to internal functionality of a module. With a package.json at the root, one can set a package.json.main or package.json.exports to explicitly tell which files and functions are part of the public interface.



1.2 Use environment aware, secure, and hierarchical config.

Consider a mandatory environment variable that wasn't provided. The app starts successfully and serve requests, some information is already persisted to DB. Then, it's realized that without this mandatory key the request can't complete, leaving the app in a dirty state.

1.3 Consider all the consequences when choosing the main framework

When building apps and APIs, using a framework is mandatory. It's easy to overlook alternative frameworks or important considerations and then finally land on a sub optimal option. As of 2023/2024, we believe that these four frameworks are worth considering: Nest.js, Fastify, Express, and Koa

2. Error Handling Practices

2.1 Use Async-Await or promises for async error handling.

Handling async errors in call-back style is probably the fastest way to hell. The best gift you can give to your code is using Promises with async-await which enables a much more compact and familiar code syntax like try-catch. OtherwiseNode.js call-back style, function (err, response), is a promising way to un-maintainable code due to the mix of error handling with casual code, excessive nesting, and awkward coding patterns.

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

Call-back hell

```
getData(someParameter, function(err, result) {
    if(err !== null) {
        // do something like calling the given callback function and pass the error
        getMoreData(a, function(err, result) {
            if(err !== null) {
                // do something like calling the given callback function and pass the error
                getMoreData(b, function(c) {
                    getMoreData(d, function(e) {
                        if(err !== null ) {
                            // you get the idea?
                        }
                    })
                });
            }
        });
    }
});
```

Promises

```
return functionA()
    .then(functionB)
    .then(functionC)
    .then(functionD)
    .catch((err) => logger.error(err))
    .then(alwaysExecuteThisFunction)
```

async – await

```
async function executeAsyncTask() {
    try {
        const valueA = await functionA();
        const valueB = await functionB(valueA);
        const valueC = await functionC(valueB);
        return await functionD(valueC);
    }
    catch (err) {
        logger.error(err);
    } finally {
        await alwaysExecuteThisFunction();
    }
}
```


© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

2.2 Use Handle errors centrally, not within a middleware

Error handling logic such as logging, deciding whether to crash and monitoring metrics should be encapsulated in a dedicated and centralized object that all entry-points (e.g. APIs, cron jobs, scheduled jobs) call when an error comes in. Otherwise Not handling errors within a single place will lead to code duplication and probably to improperly handled errors.

Avoid handling errors in the middleware.

```
// middleware handling the error directly, who will handle Cron jobs and testing
errors?
app.use((err, req, res, next) => {
   logger.logError(err);
   if (err.severity == errors.high) {
     mailer.sendMail(configuration.adminMail, 'Critical error occured', err);
   }
   if (!err.isOperational) {
     next(err);
   }
 });
```

instead, do the below.

```
module.exports.handler = new errorHandler();
function errorHandler() {
  this.handleError = async (error, responseStream) => {
    await logger.logError(error);
    await fireMonitoringMetric(error);
    await crashIfUntrustedErrorOrSendResponse(error, responseStream);
  };
}
```

```
// Error handling middleware, we delegate the handling to the centralized error handler
app.use(async (err, req, res, next) => {
   await errorHandler.handleError(err, res);//The error handler will send a response
});

process.on("uncaughtException", error => {
   errorHandler.handleError(error);
});

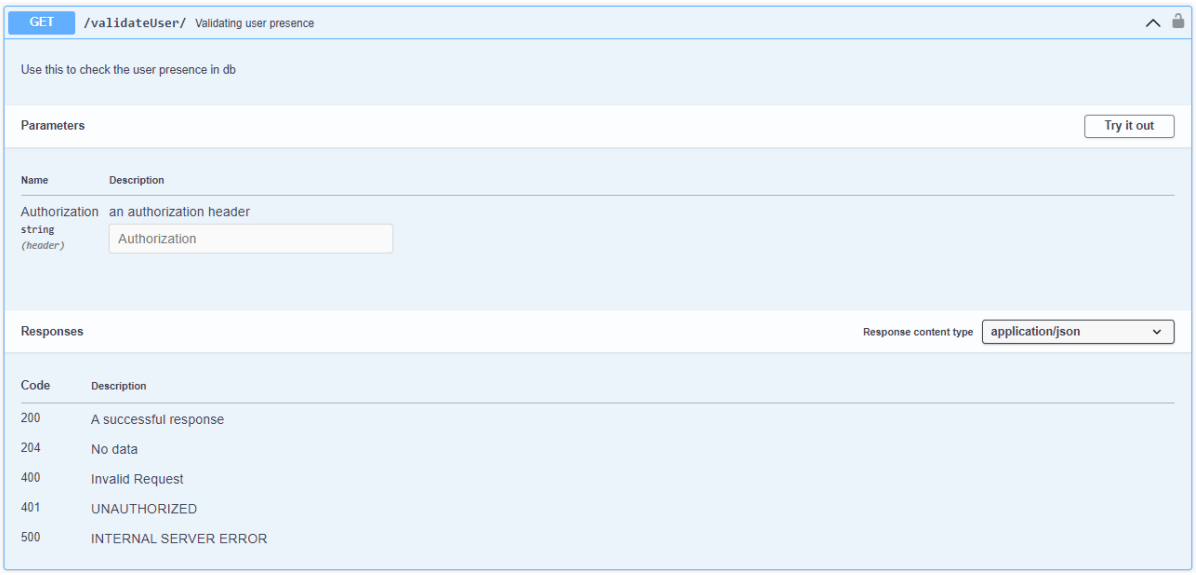
process.on("unhandledRejection", (reason) => {
   errorHandler.handleError(reason);
});
```


© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

2.3 Document API errors using OpenAPI or GraphQL

Let your API callers know which errors might come in return so they can handle these thoughtfully without crashing. For RESTful APIs, this is usually done with documentation frameworks like OpenAPI. Otherwise, An API client might decide to crash and restart only because it received back an error it couldn't understand.

REST APIs return results using HTTP status codes, it's absolutely required for the API user to be aware not only of the API schema but also of potential errors – the caller may then catch an error and tactfully handle it.



2.4 Use a mature logger to increase errors visibility

A robust logging tools like Pino or Winston increases the errors visibility using features like log-levels, pretty print colouring and more. Console.log lacks these imperative features and should be avoided. The best-in-class logger allows attaching custom useful properties to log entries with minimized serialization performance penalty. Developers should write logs to stdout and let the infrastructure pipe the stream to the appropriate log aggregator.

```
C:\Users\u290515\development\uap\api>npm start

> UAP-Node-API@8.0.0 start
> node src/server.js

ProcessExecutions(info)#2023-10-06 18:08:05 : metamorph cron job initialized : 0 1 1 * *
UAP-Node-API(8.0.0) node app is started listening on port 3000
server(info)#2023-10-06 18:08:05 : UAP-Node-API(8.0.0) node app is started listening on port 3000
Auth Middleware(error)#2023-10-06 18:08:17 : jwt expired
Exception handler(error)#2023-10-06 18:08:17 : jwt expired
Result(info)#2023-10-06 18:08:17 : unknown requested GET  /validateUser/  {}
Auth Middleware(error)#2023-10-06 18:08:17 : Error encountered in authentication of user.
app(route)#2023-10-06 18:08:17 : ::1 GET /validateUser/ 400 237kb RFR-http://localhost:3000/docs/
UA-Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/117.0.0
.0 Safari/537.36 --- 11.916 ms
```


© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

2.5 Test error flows using your favourite test framework

Whether professional automated QA or plain manual developer testing – Ensure that your code not only satisfies positive scenarios but also handles and returns the right errors. On top of this, simulate deeper error flows like uncaught exceptions and ensure that the error handler treat these. Otherwise Without testing, whether automatically or manually, you can't rely on your code to return the right errors. Without meaningful errors – there's no error handling.

2.6 Avoid giving away framework information.

The application should never reveal the framework being used for e.g "x-powered-by". Use middleware to hide this information.

2.7 Validate all inputs.

Validate all input values even if it's validated from the UI to mitigate malicious activity.

2.8 Don't block the thread.

Node.js is single-threaded. High computational activity will stall the event loop and keep other processes waiting. Avoid blocking even loop by creating worker process or using forks.

2.9 Exit Process on error.

Exit process gracefully if an unpredicted error occurs.

process.exit(1)

Always start the node application with **forever** or **pm2** to start over with a clean state. It's important to differentiate between operational (external service/component failure) vs programming (unhandled undefined errors) errors.

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

2.11 Catch unhandled promise rejections

Any exception thrown within a promise will get swallowed and discarded unless a developer didn't forget to explicitly handle it. Even if your code is subscribed to process.uncaughtException! Overcome this by registering to the event **process.unhandledRejection**. Otherwise, Your errors will get swallowed and leave no trace.

```
process.on('unhandledRejection', (reason, p) => {
   // I just caught an unhandled promise rejection,
   // since we already have fallback handler for unhandled errors (see below),
   // let throw and let him handle that
   throw reason;
});

process.on('uncaughtException', (error) => {
   // I just received an error that was never handled, time to handle it and then decide
whether a restart is needed
   errorManagement.handler.handleError(error);
   if (!errorManagement.handler.isTrustedError(error))
      process.exit(1);
});
```


© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

3. Code Style Guide

3.1 Use ESLint

ESLint is the de-facto standard for checking possible code errors and fixing code style, not only to identify nitty-gritty spacing issues but also to detect serious code anti-patterns like developers throwing errors without classification. Though ESLint can automatically fix code styles, other tools like prettier are more powerful in formatting the fix and work in conjunction with ESLint. Otherwise, Developers will focus on tedious spacing and line-width concerns and time might be wasted overthinking the project's code style.

On top of ESLint standard rules that cover vanilla JavaScript, add Node.js specific plugins like eslint-plugin-node, eslint-plugin-mocha and eslint-plugin-node-security, eslint-plugin-require, /eslint-plugin-jest and other useful rules

3.2 Start a Code block's Curly Braces on the Same Line

The opening curly braces of a code block should be on the same line as the opening statement.

```
function someFunction() {
    // code block
}

// Avoid
function someFunction()
{
    // code block
}
```

3.3 Name your functions.

Name all functions, including closures and call-backs. Avoid anonymous functions. This is especially useful when profiling a node app. Naming all functions will allow you to easily understand what you're looking at when checking a memory snapshot. Otherwise Debugging production issues using a core dump (memory snapshot) might become challenging as you notice significant memory consumption from anonymous functions.

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

3.4 Use naming conventions for variables, constants, functions, and classes.

Use lower CamelCase when naming constants, variables, and functions, Upper CamelCase (capital first letter as well) when naming classes and UPPER_SNAKE_CASE when naming global or static variables. This will help you to easily distinguish between plain variables, functions, classes that require instantiation and variables declared at global module scope. Use descriptive names but try to keep them short.

```
// for global variables names we use the const/let keyword and UPPER_SNAKE_CASE
let MUTABLE_GLOBAL = "mutable value";
const PI = 3.141592653589793;

// for class name we use UpperCamelCase
class SomeClassExample {
  // for static class properties we use UPPER_SNAKE_CASE
  static STATIC_PROPERTY = "value";
}

// for functions names we use lowerCamelCase
function doSomething() {
  // for scoped variable names we use the const/let keyword and lowerCamelCase
  const someConstExample = "immutable value";
  let someMutableExample = "mutable value";
}
```

3.5 Prefer const over let not the var

Using const means that once a variable is assigned, it cannot be reassigned. Preferring const will help you to not be tempted to use the same variable for different uses and make your code clearer. If a variable needs to be reassigned, in a for loop, for example, use let to declare it. Another important aspect of let is that a variable declared using it is only available in the block scope in which it was defined. var is function scoped, not block-scoped, and shouldn't be used in ES6 now that you have const and let at your disposal. Otherwise Debugging becomes way more cumbersome when following a variable that frequently changes.

3.6 Require modules first, not inside functions.

Require modules at the beginning of each file, before and outside of any functions. This simple best practice will not only help you easily and quickly tell the dependencies of a file right at the top but also avoids a couple of potential problems.

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

3.7 Set an explicit entry point to a module/folder.

When developing a module/library, set an explicit root file that exports the public and interesting code.

```
    get: 'Success in listing BMO'
}
const errorMessage = {
    get: 'Error in listing BMO'
};
module.exports = { successMessage, errorMessage };
  const { successMessage, errorMessage } = require("../constants/BMOConstants");
```

3.8 Use the === operator

Prefer the strict equality operator === over the weaker abstract equality operator ==. == will compare two variables after converting them to a common type. There is no type conversion in ===, and both variables must be of the same type to be equal

3.9 Use arrow function expressions (=>)

arrow functions make the code structure more compact and keep the lexical context of the root function.

```
templateHeaders.forEach(header => {
    finalMap.push(userFileData[sheet].data[0].findIndex((v) => {
        return v.toString().trim() == mapData[sheet][header].toString().trim()
    }))
});
```

4. Unit Testing

Unit Testing is a software testing method where individual units/components are tested in isolation. A unit can be described as the smallest testable part of code in an application. Unit testing is generally carried out by developers during the development phase of an application

In Node.js there are many frameworks available for running unit tests. Some of them are:

- Mocha
- Jest
- Jasmine
- AVA

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

In this code **Mocha** frame work is implemented with **Chai**

Mocha is an old and widely used testing framework for node applications. It supports asynchronous operations like callbacks, promises, and async/await. It is a highly extensible and customizable framework that supports different assertions and mocking libraries.

To install it, open command prompt and type the following command:

Installs globally

npm install mocha -g

installs in the current directory

npm install mocha --save-dev

How to use Mocha?

To use this framework in your application:

Open the root folder of your project and create a new folder called test in it.

Inside the test folder, create a new file called test.js which will contain all the code related to testing.

open package.json and add the following line in the scripts block.

```
"scripts": {
   "test": "mocha --recursive --exit "
 }
```

Example:

```
// Requiring module
const assert = require('assert');

// We can group similar tests inside a describe block
describe("Simple Calculations", () => {
 before(() => {
   console.log( "This part executes once before all tests" );
 });

 after(() => {
   console.log( "This part executes once after all tests" );
 });

 // We can add nested blocks for different tests
 describe( "Test1", () => {
   beforeEach(() => {
     console.log( "executes before every test" );
   });
```


© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

```
  it("Is returning 5 when adding 2 + 3", () => {
    assert.equal(2 + 3, 5);
  });

  it("Is returning 6 when multiplying 2 * 3", () => {
    assert.equal(2*3, 6);
  });
 });

 describe("Test2", () => {
  beforeEach(() => {
    console.log( "executes before every test" );
  });

  it("Is returning 4 when adding 2 + 3", () => {
    assert.equal(2 + 3, 4);
  });

  it("Is returning 8 when multiplying 2 * 4", () => {
    assert.equal(2*4, 8);
  });
 });
});
```

Copy the above code and paste it in the test.js file that we have created before. To run these tests, open the command prompt in the root directory of the project and type the following command: npm run test

```
C:\Windows\System32\cmd.exe

  Simple Calculations
This part executes once before all tests
    Test1
executes before every test
      √ Is returning 5 when adding 2 + 3
executes before every test
      √ Is returning 6 when multiplying 2 * 3
    Test2
executes before every test
      1) Is returning 4 when adding 2 + 3
executes before every test
      √ Is returning 8 when multiplying 2 * 4
This part executes once after all tests


  3 passing (42ms)
  1 failing

  1) Simple Calculations
       Test2
         Is returning 4 when adding 2 + 3:

     AssertionError [ERR_ASSERTION]: 5 == 4
     + expected - actual

     -5
     +4

     at Context.<anonymous> (test\api\test.js:33:14)
     at processImmediate (internal/timers.js:439:21)
```

What is Chai?

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

Chai is an assertion library that is often used alongside Mocha. It can be used as a TTD (Test Driven Development) / BDD (Behavior Driven Development) assertion library for Node.js that can be paired up with any testing framework based on JavaScript. Similar to assert.equal() statement in the above code, we can use Chai to write tests like English sentences.

To install it, open the command prompt in the root directory of the project and type the following command:

npm install chai

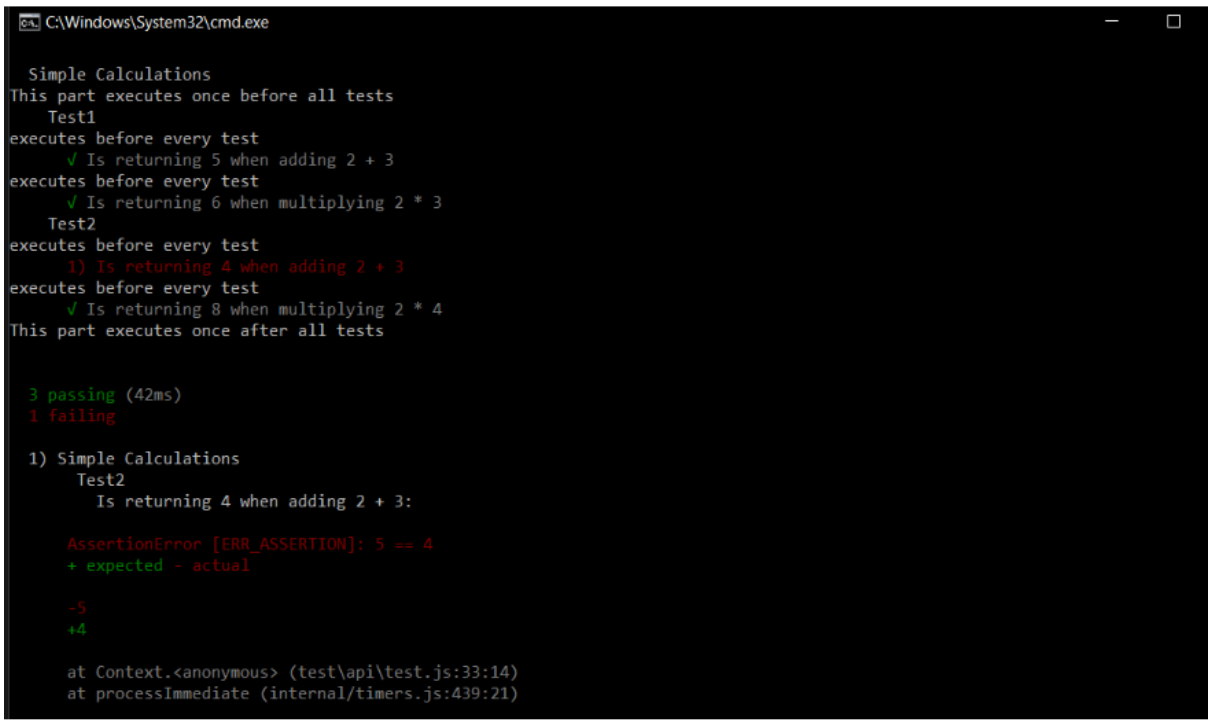
Example:

```
const expect = require('chai').expect;
describe("Testing with chai", () => {
  it("Is returning 4 when adding 2 + 2", () => {
    expect(2 + 2).to.equal(4);
  });

  it("Is returning boolean value as true", () => {
    expect(5 == 5).to.be.true;
  });

  it("Are both the sentences matching", () => {
    expect("This is working").to.equal('This is working');
  });
});
```

Output:



Code coverage tools like Istanbul/NYC are great for 3 reasons: it comes for free (no effort is required to benefit this reports), it helps to identify a decrease in testing coverage, and last but

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

not least it highlights testing mismatches: by looking at colored code coverage reports you may notice, for example, code areas that are never tested like catch clauses (meaning that tests only invoke the happy paths and not how the app behaves on errors). Set it to fail builds if the coverage falls under a certain threshold. Otherwise There won't be any automated metric telling you when a large portion of your code is not covered by testing

For Code Coverage used nyc

open package.json and add the following line in the scripts block.

```
"scripts": {
   "test": " mocha ./test/* /*.js",
   "coverage": " nyc --reporter=lcov --reporter=text npm run test"
   }
```

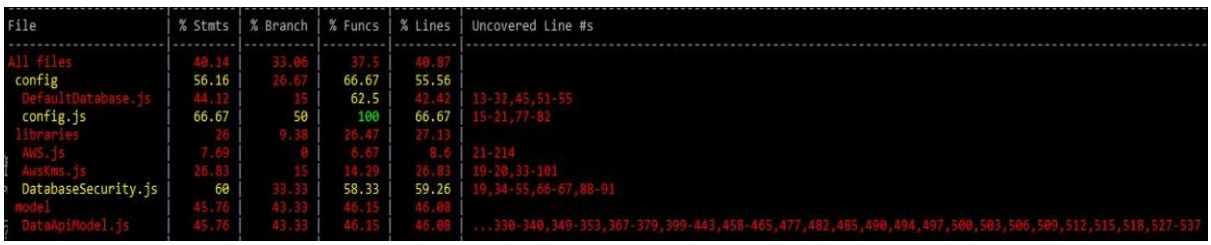
Nyc configuration added in .nycrc.json file as follows

```
{
   "include": "app",
   "check-coverage":true,
   "branches":80,
   "lines":80,
   "functions":80,
   "statements":80,
   "report-dir":"./reports"
}
```

And need to run following command

Npm run coverage

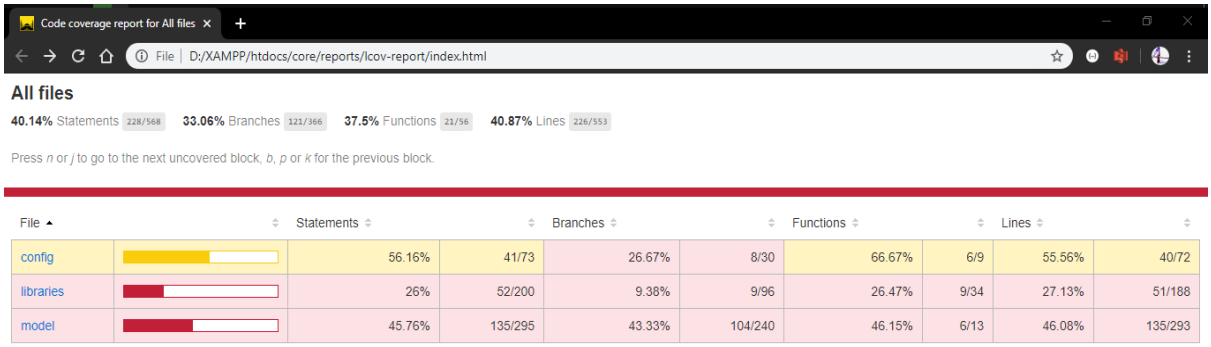
Output in command line



File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	40.14	33.06	37.5	40.87	
config	56.16	26.67	66.67	55.56	
DefaultDatabase.js	44.12	15	62.5	42.42	13-32,45,51-55
config.js	66.67	50	100	66.67	15-21,77-82
libraries	26	9.38	26.47	27.13	
AWS.js	7.69	0	6.67	8.6	21-214
AwsKms.js	26.83	15	14.29	26.83	19-20,33-101
DatabaseSecurity.js	60	33.33	58.33	59.26	19,34-55,66-67,88-91
model	45.76	43.33	46.15	46.08	
DataApiModel.js	45.76	43.33	46.15	46.08	...330-340,349-353,367-379,399-443,458-465,477,482,485,490,494,497,500,503,506,509,512,515,518,527-537

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

Output in browser: -



Unit test References

https://mochajs.org/

https://github.com/gellban/MochaChaiIstanbulNyc

https://www.npmjs.com/package/nyc

https://github.com/istanbuljs/nyc

http://codeoverflow.ebigdatai.com/codeoverflow/index.php/2018/04/22/restful-testing-and-code-coverage-using-mocha-chai-istanbul-nyc-fs-node-js-and-express/

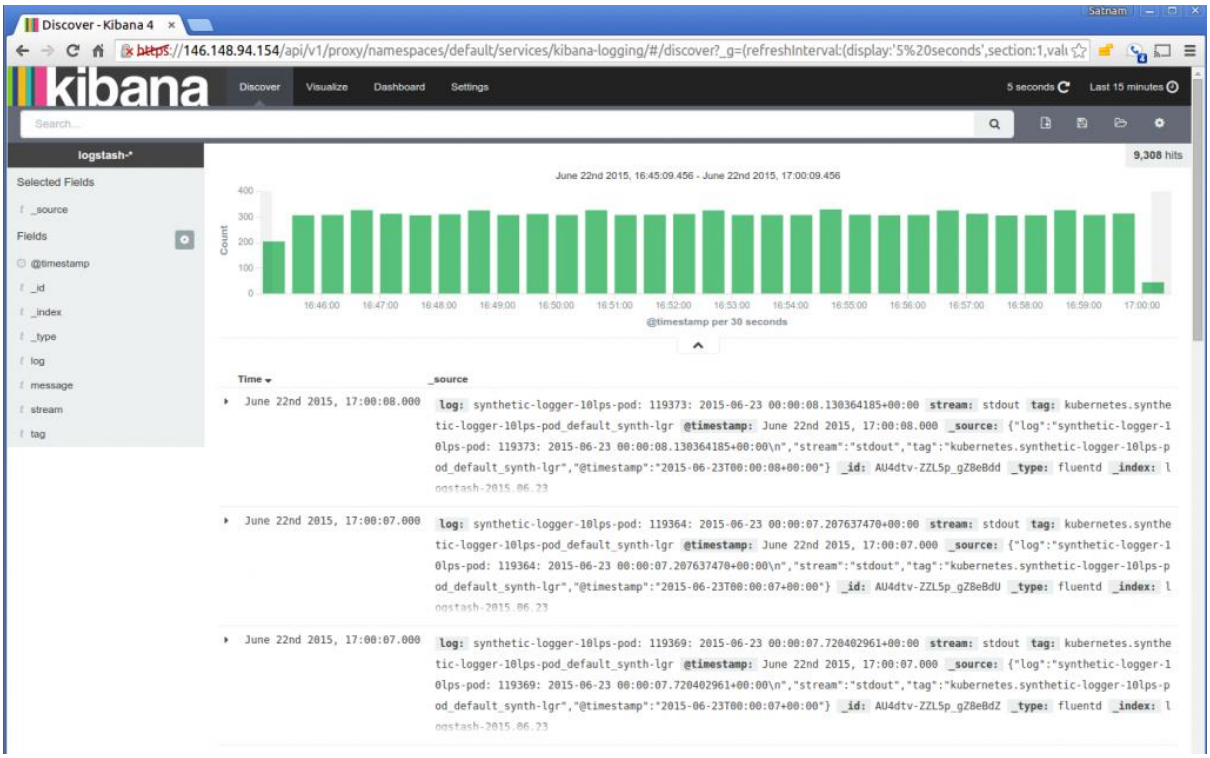
https://medium.com/@asemiloore/nodejs-testing-with-mocha-and-code-coverage-with-nyc-9d1d6e428ac1

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

5. Production deployment guidelines

5.1 Increase the observability using smart logging.

Logs can be a dumb warehouse of debug statements or the enabler of a beautiful dashboard that tells the story of your app. Plan your logging platform from day 1: how logs are collected, stored and analyzed to ensure that the desired information (e.g. error rate, following an entire transaction through services and servers, etc) can really be extracted.Use ELKStack Kibana (part of the Elastic stack) facilitates advanced searching on log content





© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

5.2 Delegate anything possible (e.g. gzip, SSL) to a reverse proxy

Node is quite bad at doing CPU intensive tasks like gzipping, SSL termination, etc. You should use specialized infrastructure like nginx, HAproxy or cloud vendor services instead. Otherwise, Your poor single thread will stay busy doing infrastructural tasks instead of dealing with your application core and performance will degrade accordingly

5.3 A Guard process uptime using the right tool

The process must go on and get restarted upon failures. Modern runtime platforms like Docker-ized platforms (e.g. Kubernetes), and Serverless take care for this automatically. When the app is hosted on a bare metal server, one must take care for a process management tools like systemd. Avoid including a custom process management tool in a modern platform that monitors an app instance (e.g., Kubernetes) - doing so will hide failures from the infrastructure. When the underlying infrastructure is not aware of errors, it can't perform useful mitigation steps like re-placing the instance in a different location. Otherwise Running dozens of instances without a clear strategy and too many tools together (cluster management, docker, PM2) might lead to DevOps chaos

5.4 Get your frontend assets out of Node.

Serve frontend content using a specialized infrastructure (nginx, S3, CDN) because Node performance gets hurt when dealing with many static files due to its single-threaded model. One exception to this guideline is when doing server-side rendering.

5.5 Use tools that automatically detect vulnerabilities

Even the most reputable dependencies such as Express have known vulnerabilities (from time to time) that can put a system at risk. This can be easily tamed using community and commercial tools that constantly check for vulnerabilities and warn (locally or at GitHub), some can even patch them immediately. Otherwise Keeping your code clean from vulnerabilities without dedicated tools will require you to constantly follow online publications about new threats. Quite tedious

5.6 Set NODE_ENV=production

Set the environment variable NODE_ENV to ‘production’ or ‘development’ to flag whether production optimizations should get activated – some npm packages determine the current environment and optimize their code for production

5.7 Design automated and zero-downtime deployments

Research shows that teams who perform many deployments lower the probability of severe production issues. Fast and automated deployments that don’t require risky manual steps and service downtime significantly improve the deployment process. You should probably achieve this using Docker combined with CI tools as they became the industry standard for streamlined deployment.

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

5.8 Use an LTS release of Node.js

Ensure you are using an LTS version of Node.js to receive critical bug fixes, security updates and performance improvements

5.9 Log to stdout, avoid specifying log destination within the app

Log destinations should not be hard-coded by developers within the application code, but instead should be defined by the execution environment the application runs in. Developers should write logs to stdout using a logger utility and then let the execution environment (container, server, etc.) pipe the stdout stream to the appropriate destination (i.e. Splunk, Graylog, ElasticSearch, etc.).

5.10 Install your packages with npm ci

Run npm ci to strictly do a clean install of your dependencies matching package.json and package-lock.json. Obviously, production code must use the exact version of the packages that were used for testing. While package-lock.json file sets strict version for dependencies, in case of mismatch with the file package.json, the command 'npm install' will treat package.json as the source of truth. On the other hand, the command 'npm ci' will exit with error in case of mismatch between these files

6. Security guidelines

6.1 Embrace linter security rules \

Make use of security-related linter plugins such as eslint-plugin-security to catch security vulnerabilities and issues as early as possible, preferably while they're being coded.

6.2 Extract secrets from config files or use packages to encrypt them.

Never store plain-text secrets in configuration files or source code. Instead, make use of secret-management systems like Vault products, Kubernetes/Docker Secrets, or using environment variables. As a last resort, secrets stored in source control must be encrypted and managed (rolling keys, expiring, auditing, etc). Make use of pre-commit/push hooks to prevent committing secrets accidentally.

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

6.3 Prevent query injection vulnerabilities with ORM/ODM libraries

To prevent SQL/NoSQL injection and other malicious attacks, always make use of an ORM/ODM or a database library that escapes data or supports named or indexed parameterized queries, and takes care of validating user input for expected types. Never just use JavaScript template strings or string concatenation to inject values into queries as this opens your application to a wide spectrum of vulnerabilities. All the reputable Node.js data access libraries (e.g. Sequelize, Knex, mongoose) have built-in protection against injection attacks. Otherwise Unvalidated or unsanitized user input could lead to operator injection when working with MongoDB for NoSQL, and not using a proper sanitization system or ORM will easily allow SQL injection attacks, creating a giant vulnerability.

6.4 Adjust the HTTP response headers for enhanced security

Your application should be using secure headers to prevent attackers from using common attacks like cross-site scripting (XSS), clickjacking and other malicious attacks. These can be configured easily using modules like helmet. Otherwise Attackers could perform direct attacks on your application's users, leading to huge security vulnerabilities

6.5 Constantly and automatically inspect for vulnerable dependencies

With the npm ecosystem it is common to have many dependencies for a project. Dependencies should always be kept in check as new vulnerabilities are found. Use tools like npm audit or snyk to track, monitor and patch vulnerable dependencies. Integrate these tools with your CI setup so you catch a vulnerable dependency before it makes it to production.

6.6 Protect Users' Passwords/Secrets using bcrypt or scrypt

Passwords or secrets (e.g. API keys) should be stored using a secure hash + salt function like bcrypt

6.7 Escape HTML, JS and CSS output

Untrusted data that is sent down to the browser might get executed instead of just being displayed, this is commonly referred as a cross-site-scripting (XSS) attack.

6.8 Validate incoming JSON schemas.

Validate the incoming requests' body payload and ensure it meets expectations, fail fast if it doesn't. To avoid tedious validation coding within each route you may use lightweight JSON-based validation.

6.9 Support blocklisting JWTs

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

When using JSON Web Tokens (for example, with Passport.js), by default there's no mechanism to revoke access from issued tokens. Once you discover some malicious user activity, there's no way to stop them from accessing the system if they hold a valid token. Mitigate this by implementing a blocklist of untrusted tokens that are validated on each request.

6.10 Hide error details from clients.

An integrated express error handler hides the error details by default. However, great are the chances that you implement your own error handling logic with custom Error objects (considered by many as a best practice). If you do so, ensure not to return the entire Error object to the client, which might contain some sensitive application details.

6.11 Modify session middleware settings.

Each web framework and technology has its known weaknesses - telling an attacker which web framework we use is a great help for them. Using the default settings for session middleware can expose your app to module- and framework-specific hijacking attacks in a similar way to the X-Powered-By header. Try hiding anything that identifies and reveals your tech stack.

6.12 A Inspect for outdated packages.

Use your preferred tool (e.g. npm outdated or npm-check-updates) to detect installed outdated packages, inject this check into your CI pipeline and even make a build fail in a severe scenario.

7. Docker guidelines

7.1 Prefer smaller Docker base images.

Large images lead to higher exposure to vulnerabilities and increased resource consumption.

7.2 Scan images for multi layers of vulnerabilities

Besides checking code dependencies vulnerabilities also scan the final image that is shipped to production. Docker image scanners check the code dependencies but also the OS binaries. This E2E security scan covers more ground and verifies that no bad guy injected bad things during the build. Consequently, it is recommended running this as the last step before deployment. There are a handful of free and commercial scanners that also provide CI/CD plugins for example trivy.

```
trivy image --security-checks vuln --exit-code 1 --severity HIGH,CRITICAL
wnsunfdanaltcsacrfc.azurecr.io/$(imageName):$(tag)
```

7.3 Clean NODE_MODULE cache

After installing dependencies in a container remove the local cache. It doesn't make any sense to duplicate the dependencies for faster future installs since there won't be any further installs

```
RUN npm ci --production && npm cache clean --force
```


© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

8. HTTP Timeouts and Resource Limits

8.1 Configure HTTP server timeouts explicitly.

Do not rely on default values, which differ between Node.js versions. Set the HTTP server timeouts according to application requirements and to the timeouts of the reverse proxy or load balancer in front of the application. When the header or request timeout expires, the server responds with 408 and closes the connection.

- **headersTimeout** – time allowed to receive the complete request headers. Keep it lower than requestTimeout.
- **requestTimeout** – time allowed to receive the entire request from the client.
- **keepAliveTimeout** – idle time the server waits for another request on a keep-alive connection. Align it with the idle timeout of the proxy or load balancer so that connections are not closed unexpectedly.
- **server.timeout** – socket inactivity timeout. It is disabled (0) by default; set it only when required.

8.2 Keep HTTP timeouts separate from application-level timeouts.

HTTP server timeouts only limit how long the server waits to receive a request or keeps an idle connection open. They do not cancel work that the application has already started. Set explicit timeouts on database queries and on calls to downstream or external APIs by using the options of the driver or client library (for example **AbortSignal.timeout()** where supported), and release the resources when a timeout occurs.

8.3 Set request body size limits per endpoint.

Set the body size limit according to each endpoint and its expected payload instead of using one large global limit. Keep a small default for JSON and form endpoints and raise it only for endpoints that need it, such as file uploads. A body size limit restricts how much data a single request can carry; it does not restrict how often a client can send requests.

8.4 Reject oversized requests and handle timeout events safely.

Reject a request whose body exceeds the limit with 413 (Payload Too Large) and stop reading the body instead of buffering it in memory. Register a **clientError** listener on the HTTP server to log malformed or timed-out requests; the listener is responsible for closing or destroying the socket.

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

```
const server = http.createServer(app);
// example values - set them per application and infrastructure
server.headersTimeout = 20000;     // receive the request headers
server.requestTimeout = 60000;     // receive the entire request
server.keepAliveTimeout = 65000;   // align with proxy / load balancer

// body size limits per endpoint (Express)
app.use(express.json({ limit: '100kb' }));
app.post('/documents',
  express.raw({ type: 'application/pdf', limit: '10mb' }), saveDocument);

server.on('clientError', (err, socket) => {
  logger.warn({ code: err.code }, 'HTTP client error');
  if (err.code === 'ECONNRESET' || !socket.writable) return;
  const status = err.code === 'ERR_HTTP_REQUEST_TIMEOUT'
    ? '408 Request Timeout' : '400 Bad Request';
  socket.end(`HTTP/1.1 ${status}\r\n\r\n`);
});
```

8.5 Limit connections and review proxy protections.

Where applicable, limit open connections and requests per connection (for example **server.maxConnections**, **server.maxRequestsPerSocket** and **maxSockets** on outbound **http.Agent** instances). Review the reverse proxy or load balancer protections (client header and body timeouts, request size limits, connection limits) against slow-client and resource-exhaustion attacks such as Slowloris, in addition to the guidance in 5.2.

8.6 Monitor, test and document the settings.

Monitor timeout and limit events, such as 408, 413 and 431 responses and **clientError** counts. Test the timeout and body-limit behaviour under normal and abnormal traffic, for example slow clients, oversized payloads and many idle connections. Document the timeout and limit values of each service, including the reverse proxy values, and keep them consistent across environments.

HTTP Timeouts and Resource Limits References
https://nodejs.org/learn/getting-started/security-best-practices
https://nodejs.org/api/http.html

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

9. Graceful Shutdown

9.1 Handle termination signals and start a controlled shutdown.

Graceful shutdown lets an application finish or stop its work in an orderly way during deployments, restarts, scaling events and controlled termination. It reduces avoidable request failures and resource leaks, but it does not guarantee that every in-flight request will succeed. Register handlers for the termination signals sent by the deployment environment (typically **SIGTERM**, and **SIGINT** for local runs or for process managers that use it) and start the shutdown only once. On non-Windows platforms, registering a listener removes the default exit behaviour of these signals, so the handler must end the process itself. Unexpected errors continue to follow 2.2, 2.9 and 2.11.

9.2 Stop accepting new requests.

When shutdown begins, stop accepting new connections with **server.close()** and report the instance as not ready (for example by failing the readiness check) so that traffic is no longer routed to it. From Node.js 19 **server.close()** also closes idle keep-alive connections; on earlier versions also call **server.closeIdleConnections()** (available from Node.js 18.2.0).

9.3 Allow in-flight requests a defined time to complete.

Give in-flight requests a defined drain period to complete. If requests are still running when the period ends, close the remaining connections with **server.closeAllConnections()** (available from Node.js 18.2.0) or let the deadline in 9.5 end the process; those requests will fail.

9.4 Close application resources cleanly.

After the server has stopped, close database connection pools, message consumers and producers, timers and scheduled jobs, and other open handles in a defined order. Stop consuming new messages first, let the work already received finish, then close the connections. Flush buffered logs and telemetry (see section 10) before the process exits.

9.5 Set a shutdown deadline and make sure the process can terminate.

Set an overall shutdown deadline that is shorter than the termination grace period of the deployment platform (for example the Kubernetes pod termination grace period, the Docker stop timeout or the kill timeout of the process manager). When that period ends the platform terminates the process forcibly and the application cannot handle it. If cleanup has not finished by the deadline, log the event and exit with a non-zero

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

code. Use a timer that does not keep the event loop alive (**timer.unref()**) so that it never delays a clean exit.

```
const SHUTDOWN_TIMEOUT_MS = Number(process.env.SHUTDOWN_TIMEOUT_MS) || 20
let shuttingDown = false;

async function shutdown(signal) {
  if (shuttingDown) return;
  shuttingDown = true;
  logger.info({ signal }, 'Shutdown started');

  // deadline: keep it shorter than the platform grace period
  const forceExit = setTimeout(() => {
    logger.error('Shutdown deadline exceeded, forcing exit');
    process.exit(1);
  }, SHUTDOWN_TIMEOUT_MS);
  forceExit.unref();

  try {
    await new Promise((resolve, reject) => {
      server.close((err) => (err ? reject(err) : resolve()));
      if (server.closeIdleConnections) server.closeIdleConnections();
    });
    await Promise.allSettled([dbPool.end(), consumer.close()]);
  } catch (err) {
    logger.error(err);
    process.exitCode = 1;
  }
  // the process exits once no handles remain, otherwise the deadline applies
}

process.on('SIGTERM', () => shutdown('SIGTERM'));
process.on('SIGINT', () => shutdown('SIGINT'));
```

9.6 Test shutdown behaviour.

Test the shutdown during application restarts and deployments. Send the termination signal while the service is handling requests and verify that new connections are refused, that in-flight requests complete or are ended within the deadline, that resources are released and that the process exits with the expected code. Include this check in the release process.

Graceful Shutdown References
https://nodejs.org/api/process.html
https://nodejs.org/api/http.html

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

10. Distributed Tracing and Standardized Metrics

10.1 Understand the role of logging, tracing and metrics.

Logs, traces and metrics answer different questions and are used together; none of them replaces the others. Logging is covered in 2.4, 5.1 and 5.9.

- **Logging** records individual events.
- **Tracing** follows a request across services and operations.
- **Metrics** quantify system behaviour over time.

10.2 Use OpenTelemetry-compatible instrumentation for distributed or business-critical service

Capture request traces and relevant service metrics with OpenTelemetry-compatible instrumentation (the OpenTelemetry JavaScript SDK and its Node.js instrumentation libraries). In the OpenTelemetry JavaScript documentation, tracing and metrics are stable signals while logs are still in development; continue to write logs as described in 5.9 and verify the current status before depending on the logs signal. Use releases that support the project's Node.js version, pin the package versions, and initialize the SDK before the application code is loaded so that the instrumentation libraries can attach. Instrumentation provides the data needed to diagnose performance problems; it does not fix them by itself.

```
// instrumentation.js - start with: node --require ./instrumentation.js app.js
const { NodeSDK } = require('@opentelemetry/sdk-node');
const {
  getNodeAutoInstrumentations,
} = require('@opentelemetry/auto-instrumentations-node');
const {
  PeriodicExportingMetricReader,
} = require('@opentelemetry/sdk-metrics');
const {
  OTLPTraceExporter,
} = require('@opentelemetry/exporter-trace-otlp-http');
const {
  OTLPMetricExporter,
} = require('@opentelemetry/exporter-metrics-otlp-http');

// endpoint and service name are read from the OTEL_* environment variables
const sdk = new NodeSDK({
  traceExporter: new OTLPTraceExporter(),
  metricReader: new PeriodicExportingMetricReader({
    exporter: new OTLPMetricExporter(),
  }),
  instrumentations: [getNodeAutoInstrumentations()],
});
```


© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

```
sdk.start();
module.exports = { sdk };  // lets the app call sdk.shutdown() (see section 9)
```

10.3 Propagate trace context and correlate it with logs.

Propagate the trace context (the W3C **traceparent** header, which is the OpenTelemetry default) across service boundaries. The HTTP instrumentation libraries do this automatically for supported clients and servers; for other transports, such as message queues, propagate the context explicitly where supported. Include the trace ID and span ID in application log entries so that logs and traces can be correlated where the observability platform supports it.

10.4 Monitor key service indicators and define thresholds.

Monitor request latency, error rate and request throughput of each service. Report latency as percentiles instead of averages and use low-cardinality attributes, such as the route template, instead of raw URLs or user identifiers. Define service-level thresholds and alerts based on the requirements of the application (for example an agreed latency target and an acceptable error rate) and review them regularly.

10.5 Do not collect secrets or unnecessary personal data in telemetry.

Do not record passwords, tokens, credentials, authorization headers, cookies, request bodies or unnecessary personal information in spans, metric attributes, baggage or logs. Review what the instrumentation captures by default (for example URLs with query strings, headers and database statements) and redact or disable anything sensitive, in line with 6.2 and 6.10.

10.6 Apply observability requirements proportionately.

Apply this section to distributed or business-critical services. Distributed tracing is not required for small or standalone applications; decide the level of observability based on the complexity and business criticality of the application. Configure the exporter endpoint and service name through the environment (for example **OTEL_EXPORTER_OTLP_ENDPOINT** and **OTEL_SERVICE_NAME**) instead of hard-coding them, and call **sdk.shutdown()** as the last step of the graceful shutdown in section 9.

Distributed Tracing and Metrics References
https://opentelemetry.io/docs/languages/js/

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

3. Application Pre-requisite

Hardware Requirements

- Windows 10, 4 GB Ram, 2 CPU, > 1 GB storage
- Linux, 4 GB RAM, 2 CPU, > 1GB storage

Software Requirements

- Node.js – latest stable version (v16)
- VSCode – latest stable version
- Git Client / Git Bash – latest stable version

Browser Support

List of browsers supported by the application

Chrome	Latest
Edge	Latest
Firefox	Latest
Safari	Latest

VS Code extensions

Listing few good to have extensions while working with

- SonarLint: Linter to detect & fix coding issues locally in JS/TS, Python, PHP, Java, C, C++, Go, IaC. Use with SonarQube & Sonar Cloud for optimal team performance.
- ESLint: This extension is a linter for ECMAScript/JavaScript code in your workspace. What it does is identify and report patterns in your code to create a codebase that is more consistent and less buggy
- Prettier: It enforces a consistent style by parsing your code and re-printing it with its own rules that take the maximum line length into account, wrapping code when necessary

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved



CO-CREATE TO OUTPERFORM

This presentation and any files attached and/or transmitted with it are confidential and intended solely for the use of the individual or entity to whom they are addressed. No part of this presentation may be given, lent, resold, or disclosed to any unintended recipients or exploited for any commercial purposes. If you are not the intended recipient and you have received this presentation in error, please return this material to the sender immediately and forthwith delete and destroy the presentation including any copies thereof from your records. We hereby notify that disclosing, distributing, copying, reproducing, storing in a retrieval system, or transmitting in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, or taking any action in reliance on the contents of the presentation in its entirety or any part thereof is strictly prohibited without the prior written consent of WNS, such consent being given at the sole discretion of WNS. Any views or opinion expressed in this presentation are those of the author and do not necessarily represent that of WNS. WNS makes no representations and to the full extent permissible by applicable law, WNS disclaims any warranties of any kind, express or implied, including any warranty of merchantability, accuracy, fitness or applicability for a particular purpose, and non-infringement of third party rights, as to the information, content and materials.

WNS